

MSBA Workshop: Linear Regression and Matrix Algebra

Kevin Ross

Jul 9, 2026

Linear Regression

Suppose we want to predict (or estimate) the sale price (y) of a house based on its square footage (x). Linear regression provides one way to do this by “fitting a line” to the data. That is, our predictions ($\hat{y}$) will be of the form

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

where $\hat{\beta}_0$ is the *intercept* and $\hat{\beta}_1$ is the *slope* or *coefficient* for square footage.

We’ll use the Ames housing data as an example; **Gr Liv Area** (x) is square footage and **SalePrice** (y) is measured in 2010 US dollars.

```
import pandas as pd

import numpy as np

df_ames = pd.read_csv("http://dlsun.github.io/pods/data/AmesHousing.txt", sep = "\t")

df = df_ames[["Gr Liv Area", "SalePrice"]]

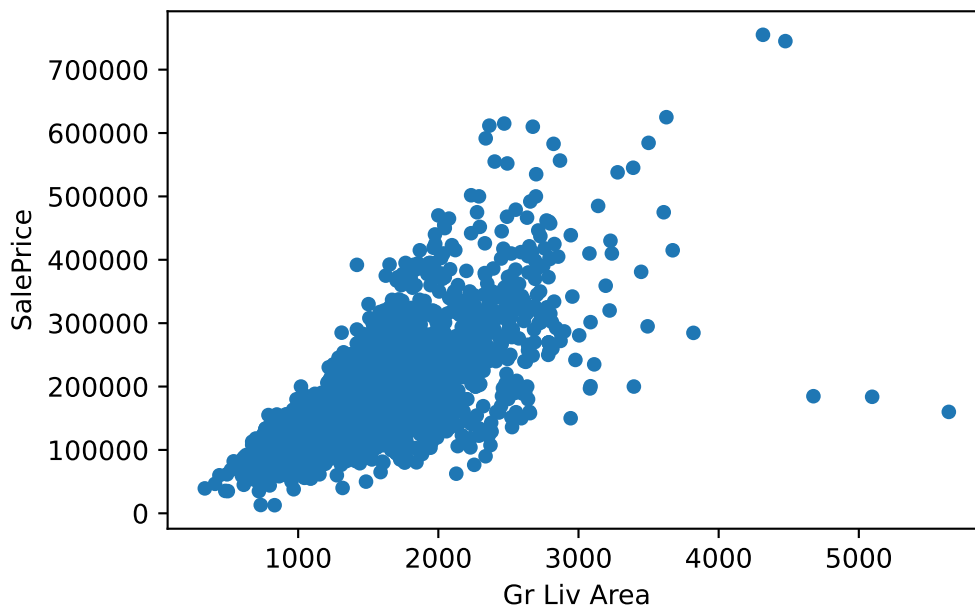
df
```

	Gr Liv Area	SalePrice
0	1656	215000
1	896	105000
2	1329	172000

	Gr Liv Area	SalePrice
3	2110	244000
4	1629	189900
...	...	...
2925	1003	142500
2926	902	131000
2927	970	132000
2928	1389	170000
2929	2000	188000

First we summarize the data.

```
df.plot.scatter(x="Gr Liv Area", y="SalePrice")
```



The number of observations, n

```
len(df)
```

2930

The mean of each variable, $\bar{x}$, $\bar{y}$

```
df[["Gr Liv Area", "SalePrice"]].mean()
```

```
Gr Liv Area      1499.690444
SalePrice        180796.060068
dtype: float64
```

The standard deviation of each variable, s_x , s_y .

```
df[["Gr Liv Area", "SalePrice"]].std()
```

```
Gr Liv Area      505.508887
SalePrice        79886.692357
dtype: float64
```

The correlation between the variables r

```
df[["Gr Liv Area", "SalePrice"]].corr()
```

	Gr Liv Area	SalePrice
Gr Liv Area	1.00000	0.70678
SalePrice	0.70678	1.00000

Linear regression is one *machine learning* technique. Machine learning in Python is usually performed using the **sciKit-Learn** package. This library not only provides convenient functions for fitting machine learning models, it also enforces a strict structure of workflow that will help you make responsible choices. (You'll cover machine learning and **scikit-learn** in much more detail in GSB 544.)

```
from sklearn.linear_model import LinearRegression
```

```
sqft_model = LinearRegression()
sqft_model.fit(
    X=df[["Gr Liv Area"]],
    y=df["SalePrice"]
)
```

```
LinearRegression()
```

The above code “fits the line” using the Ames data. The fitted slope (`coef_`) and intercept (`intercept_`) are

```
sqft_model.coef_, sqft_model.intercept_
```

```
(array([111.69400086]), 13289.63436475952)
```

So our prediction equation is

$$\hat{y} = 13289 + 111.694x$$

The predicted sale price for a 3000 sqft house is

$$\hat{y} = 13289 + 111.694 \times 3000 = 348371$$

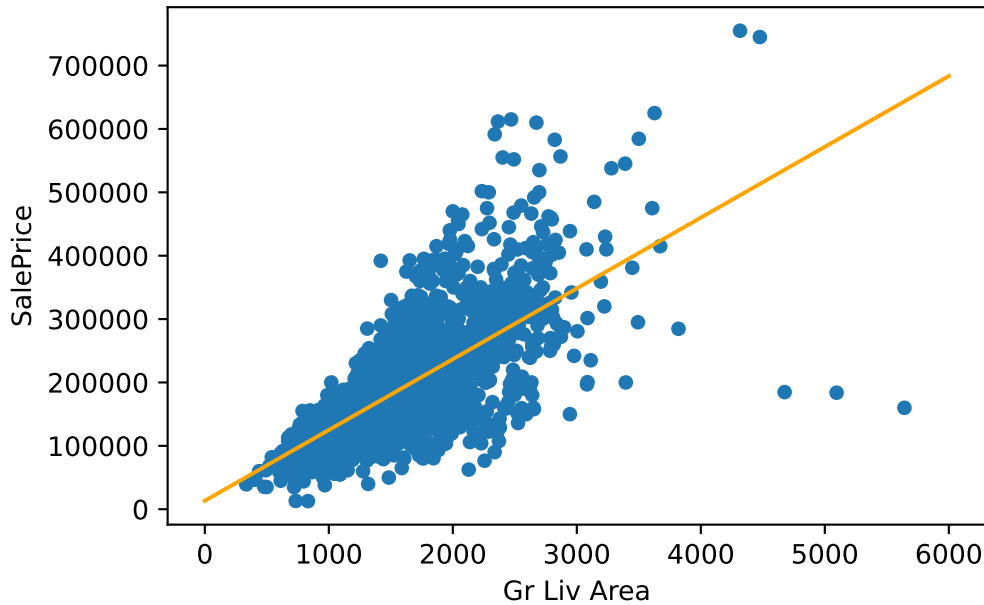
```
sqft_model.predict([[3000]])
```

```
array([348371.63693816])
```

The following code computes the predicted value for $x = 0, 1, 2, \dots, 5999, 6000$ and plots the predicted values on the original scatter plot. We see that the predicted values all lie along a straight line.

```
X_new = pd.DataFrame()
X_new["Gr Liv Area"] = np.linspace(0, 6000, num=6000)
y_new_ = pd.Series(
    sqft_model.predict(X_new),
    index=X_new["Gr Liv Area"]
)

df.plot.scatter(x="Gr Liv Area", y="SalePrice")
y_new_.plot.line(c="orange")
```



Least Squares

In the previous section we fitted a line to the data, resulting in a line with a specific slope and intercept. But why that particular line instead of some other line? How do we know which line is “best”?

A line is defined by its slope and intercept. Given a particular slope β_1 and intercept β_0 , the predicted value for a certain x will be $\beta_0 + \beta_1 x$. For the observed (x_i, y_i) values in the data set, we can compare the actual value y_i and the predicted value $\beta_0 + \beta_1 x_i$; the difference is the “prediction error” or *residual* and is represented on the scatterplot by the vertical distance between the actual value and the predicted value. The *least squares* criteria says to choose the line that minimizes the sum of the squared residuals for all the observations

$$S(\beta_0, \beta_1) = \sum_i (y_i - (\beta_0 + \beta_1 x_i))^2$$

To see the idea of least squares [try this applet](#):

- There is some data loaded in (context doesn’t matter, but it’s predicting height from shoe size)
- Click on “Show Movable Line”. A horizontal line should appear on the scatter plot. This line predicts the same value of y (the observed mean $\bar{y}$) regardless of x
- Click on “Show Residuals”. You should now see the vertical distances on the plot. These represent the prediction errors if you predicted the same value of y regardless of x .

- Now click and drag one of the green boxes on the end of the line to change its slope, and click and drag the green box in the middle to move the line up and down (changing the intercept). Notice how “Sum Abs Errors (SAE)” changes as you move the line around
- Least squares minimizes the sum of the *squared* residuals (rather than absolute). Click on the box for “Show Squared Residuals” and you’ll see a bunch of squares on the plot representing the squared residuals.
- Move the line around and observed how the squared residuals and “Sum Squared Errors (SSE)” changes.
- Keep moving the line around until you have made the SSE as small as possible.
- Now click on the box for “Show Regression Line” which will display the least squares regression line. How close was your line to the regression line?
- Under “Show Regression Line”, click on the box for “Show Squared Residuals”. Continue to move your line around; can you beat the SSE for the regression line?

Least squares regression can be set up as a [calculus minimization problem](#).

The slope and intercept of the least squares regression line are

$$\hat{\beta}_1 = \frac{rs_y}{s_x}$$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

Linear Regression Model

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i$$

- β_1 : “true” slope
- β_0 : “true” intercept
- ϵ_i : “true” residual, assumed to have some distribution (usually Normal with mean 0 and constant variance)

The line $\beta_0 + \beta_1 x$ represents how the *mean* of y varies with x ; ϵ represents how the actual y values vary about their mean.

What’s the difference between the “model” and the “fitted model”?

- The model is defined by some “true” slope β_1 and intercept β_0 which represents what we would see if we had data on a large population of many, many observations. The values β_0 and β_1 are *unknown parameters*.
- The fitted model estimates the unknown parameters β_1 and β_0 based on the observed sample data. The fitted intercept $\hat{\beta}_0$ and slope $\hat{\beta}_1$ are *observed statistics* computed from the sample data.

You might encounter formulas like the following when dealing with linear regression; what does it all mean?

$$\begin{aligned}\mathbf{Y} &= \mathbf{X}\beta + \epsilon \\ \hat{\mathbf{Y}} &= \mathbf{X} \hat{\beta} \\ \hat{\beta} &= (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}\end{aligned}$$

To understand the above, we need a little matrix algebra. We only introduce what we need in the context of a tabular data set with rows representing observations and columns representing variables.

Vector

You can think of a vector as an ordered list of values. For example, a vector might be a row/observation in a data set; the vector $[1656, 215000]$ represents a house with 1656 square feet and a 215000 sale price. Most often we think of vectors as columns in a data set; the `SalePrice` vector has 2930 elements $\mathbf{y} = [2150000, 10500, 172000, \dots, 188000]$.

Vectors can either be row vectors or column vectors, but we usually think of them as columns. If $\mathbf{x}$ is a column vector then $\mathbf{x}^T$ is a row vector.

Vectorization

Operations on vectors are performed element-by-element

$$10 \times \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 10 \times 1 \\ 10 \times 2 \\ 10 \times 3 \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} + \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 1 + 4 \\ 2 + 5 \\ 3 + 6 \end{bmatrix}$$

Vector dot product

Suppose your course grade is based on scores $\mathbf{x} = [100, 70, 80, 90]$ with weights $\mathbf{y} = [0.1, 0.2, 0.3, 0.4]$. Your weighted course average is

$$0.1 \times 100 + 0.2 \times 70 + 0.3 \times 80 + 0.4 \times 90 = 84$$

This is an example of a vector dot product, like a sumproduct, of two vectors with the same length

$$\mathbf{x} \cdot \mathbf{y} = \sum_i x_i y_i$$

If $\mathbf{x}$ and $\mathbf{y}$ are both column vectors of the same length, the dot product can be written as $\mathbf{x}^T \mathbf{y}$ (see more on matrix multiplication below).

```
import numpy as np

x = np.array([100, 70, 80, 90])
y = np.array([0.1, 0.2, 0.3, 0.4])

np.dot(x, y)
```

84.0

Matrix

A matrix is a rectangular array of values. We often think of a data frame as a matrix with one row for each observation and one column for each variable. For matrix $\mathbf{A}$, a_{ij} denotes the entry in row i and column j . An $n \times p$ matrix has n rows and p columns.

A column vector can be considered an $n \times 1$ matrix, and a row vector as a $1 \times p$ matrix.

A square matrix has the same number of rows and columns, $n \times n$.

Matrix Transpose

To find the transpose of a matrix, flip the rows and columns. If $\mathbf{A}$ is an $n \times p$ matrix then $\mathbf{A}^T$ is an $p \times n$ matrix.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Matrix multiplication

Continuing the weighted course average example above, suppose we have scores for three students (each student is a different row; each column is a different grade item)

$$\mathbf{X} = \begin{bmatrix} 100 & 70 & 80 & 90 \\ 95 & 75 & 80 & 100 \\ 90 & 90 & 80 & 80 \end{bmatrix}$$

with weights

$$\mathbf{Y} = \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.4 \end{bmatrix}$$

To find the weighted average for each student, we take the row corresponding to the student and dot product it with the weight vector, resulting in a vector with 3 rows and 1 column.

$$\mathbf{XY} = \begin{bmatrix} 0.1 \times 100 + 0.2 \times 70 + 0.3 \times 80 + 0.4 \times 90 \\ 0.1 \times 95 + 0.2 \times 75 + 0.3 \times 80 + 0.4 \times 100 \\ 0.1 \times 90 + 0.2 \times 90 + 0.3 \times 80 + 0.4 \times 80 \end{bmatrix} = \begin{bmatrix} 84 \\ 88.5 \\ 83 \end{bmatrix}$$

This is an example of matrix multiplication.

Two matrices $\mathbf{A}$ and $\mathbf{B}$ can only be multiplied if their dimensions align; the matrix product $\mathbf{AB}$ can only be computed if the number of columns of $\mathbf{A}$ is the same as the number of rows of $\mathbf{B}$.

If $\mathbf{A}$ is an $n \times k$ matrix and $\mathbf{B}$ is an $k \times p$ matrix then the matrix product $\mathbf{AB}$ is an $n \times p$ matrix where the ij entry is the dot product of the i th row of $\mathbf{A}$ and the j column of $\mathbf{B}$

```
X = np.array([
    [100, 70, 80, 90],
    [95, 75, 80, 100],
    [90, 90, 80, 80]])

y = np.array([0.1, 0.2, 0.3, 0.4])

X @ y
```

```
array([84. , 88.5, 83. ])
```

Identity matrix

The identity matrix is a square matrix with 1s on the diagonals and 0 everywhere else.

For example, the 3×3 identity matrix is

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

If $\mathbf{A}$ is $n \times n$ and $\mathbf{I}$ is $n \times n$ then

$$\mathbf{AI} = \mathbf{IA} = \mathbf{A}$$

Matrix inverse

An $n \times n$ square matrix $\mathbf{A}$ is *invertible* if there exists a $n \times n$ square matrix $\mathbf{A}^{-1}$, called the *matrix inverse*, which satisfies

$$\mathbf{AA}^{-1} = \mathbf{I} = \mathbf{A}^{-1}\mathbf{A}$$

If the inverse exists then it is unique, but some matrices do not have an inverse.

See Wikipedia for an [example](#)

In `numpy`, the inverse of a matrix $\mathbf{A}$ is computed with `np.linalg.inv(A)`.

The Design Matrix

Now we have enough matrix vocabulary to make sense of

$$\mathbf{Y} = \mathbf{X}\beta + \epsilon$$

$\mathbf{Y}$ is just the column vector of the n observed values of the response variable, e.g. the 2930 `SalePrice` values.

$\mathbf{X}$, called the *design matrix*, has one row per observation and one column per predictor, **plus a column of all 1s for the intercept**. For simple linear regression with a single predictor (square footage), $\mathbf{X}$ is $n \times 2$:

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}, \quad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}$$

Multiplying out $\mathbf{X}\beta$ gives a column vector whose i th entry is $\beta_0 + \beta_1 x_i$ — exactly the regression equation for observation i , for every observation at once. This is the same idea as the weighted-course-average example above, just with a “1” column mixed in to produce the intercept.

The *fitted/predicted* values use the estimated coefficients $\hat{\beta}$ in place of β

$$\hat{\mathbf{Y}} = \mathbf{X} \hat{\beta}$$

and the *least squares* estimates — the values of $\hat{\beta}$ that minimize the sum of squared residuals, generalizing the $S(\beta_0, \beta_1)$ from before to any number of predictors — turn out to have a closed-form matrix solution called the **normal equations**

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$$

We won’t derive this (it comes from calculus — take the derivative of $S(\beta)$ with respect to the vector β , set it to $\mathbf{0}$, and solve), but we can check that it works.

Checking the formula: simple linear regression

Let’s build $\mathbf{X}$ and $\mathbf{Y}$ for the square-footage example and compute $\hat{\beta}$ directly with the matrix formula.

```

X = np.column_stack([
    np.ones(len(df)),
    df["Gr Liv Area"]
])

Y = df["SalePrice"]

beta_hat = np.linalg.inv(X.T @ X) @ X.T @ Y
beta_hat

```

```
array([13289.63436476, 111.69400086])
```

Compare this to `sqft_model.intercept_` and `sqft_model.coef_` computed earlier using `scikit-learn`: they match. `scikit-learn` isn't doing anything magical — for ordinary linear regression it's solving the same normal equations under the hood.

Multiple Regression

Nothing about the design matrix or the normal equations above was specific to having just *one* predictor. This is the payoff for learning the matrix version: the exact same formula, $\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$, works no matter how many columns (predictors) $\mathbf{X}$ has. There is no separate “multiple regression formula” to learn — the slope/intercept shortcut formulas from before only worked for a single predictor, but the matrix formula scales to any number of predictors for free.

Suppose we predict `SalePrice` from *both* `Gr Liv Area` and `Year Built`. The design matrix just gets an extra column:

$$\mathbf{X} = \begin{bmatrix} 1 & x_{1,1} & x_{1,2} \\ 1 & x_{2,1} & x_{2,2} \\ \vdots & \vdots & \vdots \\ 1 & x_{n,1} & x_{n,2} \end{bmatrix}, \quad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

First with `scikit-learn`:

```

df_multi = df_ames[["Gr Liv Area", "Year Built", "SalePrice"]]

multi_model = LinearRegression()
multi_model.fit(
    X=df_multi[["Gr Liv Area", "Year Built"]],
    y=df_multi["SalePrice"]
)

```

```
)
```

```
multi_model.intercept_, multi_model.coef_
```

```
(-2106459.470000774, array([ 95.96949261, 1087.23673336]))
```

Now the matrix version, by hand, adding a column for Year Built to $\mathbf{X}$:

```
X_multi = np.column_stack([
    np.ones(len(df_multi)),
    df_multi["Gr Liv Area"],
    df_multi["Year Built"]
])

Y_multi = df_multi["SalePrice"]

beta_hat_multi = np.linalg.inv(X_multi.T @ X_multi) @ X_multi.T @ Y_multi
beta_hat_multi
```

```
array([-2.10645947e+06,  9.59694926e+01,  1.08723673e+03])
```

Same answer, same formula, one more column. This is one reason why coefficients in multiple regression are interpreted “holding the other variables constant”: each $\hat{\beta}_j$ is estimated jointly with all the others, not one at a time. You’ll see multiple regression again (and this same matrix notation) in your coursework.